[Q] Module 06 -- 120 Technical Interview Questions Bank

CONFIDENTIAL & PRIVATE STUDY MATERIAL
Target Roles: AI Engineer, Applied AI Engineer, Backend AI Engineer
<- Back to Study Notes Index - Previous Module: Security & Reliability <- - Next Module: Interview Answers in Hinglish ->

[INDEX] Categorized Question Index

This question bank contains exactly **120 high-yield interview questions** structured across 15 technical engineering domains. Every question is grounded in the Hirely codebase and progresses from foundational fundamentals to advanced production trade-offs.

- Category 01: Project Introduction & Problem-Solving (Q1 - Q8)
- Category 02: Python & Backend Engineering (Q9 - Q16)
- Category 03: FastAPI & REST APIs (Q17 - Q24)
- Category 04: Authentication & Authorization (Q25 - Q32)
- Category 05: PostgreSQL, SQLAlchemy & Migrations (Q33 - Q40)
- Category 06: Resume Parsing & Document Processing (Q41 - Q48)
- Category 07: Machine Learning & Matching Logic (Q49 - Q56)
- Category 08: LLMs & Google Gemini Integration (Q57 - Q64)
- Category 09: RAG, Embeddings, Retrieval & Grounding (Q65 - Q72)
- Category 10: Prompt Engineering & Output Validation (Q73 - Q80)
- Category 11: AI Evaluation, Latency, Tokens & Cost (Q81 - Q88)
- Category 12: Security & Reliability Engineering (Q89 - Q96)
- Category 13: Docker & Container Orchestration (Q97 - Q104)
- Category 14: Testing, Debugging & Trade-Offs (Q105 - Q112)
- Category 15: Limitations, Technical Debt & Ownership (Q113 - Q120)

Category 01: Project Introduction & Problem-Solving (Q1 - Q8)

- **Q001:** What is Hirely, and what core business problem does it solve in modern recruitment?
- **Q002:** Why did you design Hirely as a dual-sided platform instead of a candidate-only career tool?
- **Q003:** Why is conventional keyword-based ATS filtering flawed, and how does Hirely address it?
- **Q004:** What is the fundamental difference between a naive LLM wrapper and Hirely's hybrid architecture?
- **Q005:** Walk me through the candidate application journey from signup to application tracking.
- **Q006:** Walk me through the recruiter hiring journey from job creation to candidate intelligence.
- **Q007:** What were the most critical engineering trade-offs you made during the development of Hirely?
- **Q008:** If you had to pitch Hirely in 60 seconds to a CTO, how would you articulate its technical value?

Category 02: Python & Backend Engineering (Q9 - Q16)

- **Q009:** Why did you choose Python 3.12 for Hirely, and what Python-specific features did you utilize?
- **Q010:** How does Python's `contextvars` module enable asynchronous request isolation in Hirely?
- **Q011:** What is the difference between synchronous (`def`) and asynchronous (`async def`) endpoints in FastAPI, and how did you decide which to use?
- **Q012:** How are Python type hints and Pydantic v2 used to enforce data integrity across service boundaries?
- **Q013:** How did you structure the backend to follow the Separation of Concerns principle?
- **Q014:** How does Hirely manage thread safety in its in-memory rate limiting and metrics collections?
- **Q015:** What is the GIL (Global Interpreter Lock), and how does it affect CPU-bound tasks like document parsing?
- **Q016:** How are Python generators utilized in FastAPI database session dependency management?

Category 03: FastAPI & REST APIs (Q17 - Q24)

- **Q017:** Why did you select FastAPI over Django or Flask for the Hirely backend?
- **Q018:** How does FastAPI's Dependency Injection system work, and which dependencies did you create?
- **Q019:** How does middleware execution order work in ASGI applications, and what is Hirely's middleware stack?
- **Q020:** How do you handle path parameters, query parameters, and JSON request bodies in FastAPI routes?
- **Q021:** How does Hirely implement HTTP status code discipline (e.g., 200, 201, 400, 403, 404, 409, 502)?

- **Q022:** What is the purpose of the `/health` and `/health/db` endpoints, and how do they differ?
- **Q023:** How does FastAPI automatically generate OpenAPI (Swagger) documentation, and how is it customized?
- **Q024:** How do you prevent request latency spikes when multiple slow external AI requests occur concurrently?

Category 04: Authentication & Authorization (Q25 - Q32)

- **Q025:** Explain Hirely's authentication flow from user registration to accessing protected routes.
- **Q026:** Why did you use Argon2 for password hashing instead of bcrypt or SHA-256?
- **Q027:** What claims are encoded in Hirely's JWT, and what is their cryptographic significance?
- **Q028:** How is stateless authentication verified on every incoming protected request?
- **Q029:** What happens when an expired JWT token is passed to an authenticated endpoint?
- **Q030:** How is Role-Based Access Control (RBAC) enforced between candidates and recruiters?
- **Q031:** What prevents a candidate from tampering with another candidate's profile or resume?
- **Q032:** Where are JWT signing secrets stored, and what are the security risks of hardcoding them?

Category 05: PostgreSQL, SQLAlchemy & Migrations (Q33 - Q40)

- **Q033:** Why did you choose PostgreSQL over a NoSQL database like MongoDB for Hirely?
- **Q034:** Explain the relational schema design: How are Users, Candidates, Recruiters, and Companies linked?
- **Q035:** How does the database enforce that a candidate cannot apply to the same job opening twice?
- **Q036:** What is the difference between SQLAlchemy ORM and raw SQL queries, and why use SQLAlchemy 2.0?
- **Q037:** How does database connection pooling work, and how did you budget pool sizing for multiple workers?
- **Q038:** What are `pool_pre_ping` and `pool_recycle`, and what real-world network issues do they prevent?
- **Q039:** Why are schema migrations essential, and how does Alembic track database revision history?
- **Q040:** How does the Docker Compose setup guarantee that database migrations run before the backend starts?

Category 06: Resume Parsing & Document Processing (Q41 - Q48)

- **Q041:** What is Docling, and why did you choose it over standard PDF parsers like PyPDF2 or pdfminer?
- **Q042:** How does Docling handle complex multi-column resume layouts and visual document hierarchy?
- **Q043:** Walk me through the resume upload pipeline from file receipt to active resume selection.
- **Q044:** How does the backend extract and normalize candidate skills from raw parsed resume text?
- **Q045:** How are resume skills synchronized into the candidate's primary skill profile?
- **Q046:** Why is document parsing CPU-intensive, and what are the architectural risks of running it synchronously?
- **Q047:** What security checks are performed on uploaded PDF files before passing them to the parser?
- **Q048:** How does the system handle corrupt, password-protected, or unreadable PDF documents?

Category 07: Machine Learning & Matching Logic (Q49 - Q56)

- **Q049:** Explain the mathematical formula used by Hirely's Hybrid ATS Resume Analyzer.
- **Q050:** Why are Required Skills weighted at 45% while Preferred Skills are weighted at 15%?
- **Q051:** What is the Resume Completeness score (10%), and what specific heuristics does it measure?
- **Q052:** What is the difference between exact keyword matching and semantic vector similarity?
- **Q053:** How does the Recruiter Candidate Matching Engine rank an applicant pool for a specific job opening?
- **Q054:** How does Hirely generate qualitative match explanations (strengths, risks, interview questions)?
- **Q055:** How does candidate profile skill data combine with active parsed resume skills during evaluation?
- **Q056:** Why did you decide against letting the LLM calculate the numerical match score directly?

Category 08: LLMs & Google Gemini Integration (Q57 - Q64)

- **Q057:** Why did you select Google Gemini (`gemini-2.5-flash`) as the primary generative model?
- **Q058:** How is the `google-genai` Python SDK integrated into Hirely's backend architecture?
- **Q059:** What is Hirely's automated model fallback strategy when upstream API outages (HTTP 503) occur?
- **Q060:** How does Hirely implement exponential backoff retry logic for transient AI failures?
- **Q061:** How do you enforce structured JSON responses from Gemini instead of free-form conversational text?
- **Q062:** How does the AI Resume Improvement feature generate targeted before-and-after bullet rewrites?
- **Q063:** How does the AI Cover Letter Generator prevent fabricating experience not present in the resume?
- **Q064:** What is the role of the AI Job Description Analyzer, and how does it assist recruiters before posting?

Category 09: RAG, Embeddings, Retrieval & Grounding (Q65 - Q72)

- **Q065:** What is Retrieval-Augmented Generation (RAG), and why is it needed in Hirely?
- **Q066:** Walk me through the architecture of the Candidate Resume Evidence Assistant.
- **Q067:** How are resumes chunked before embedding, and why is chunk size and overlap important?
- **Q068:** Which embedding model is used, what is its vector dimensionality, and how is it invoked?
- **Q069:** How is Cosine Similarity calculated between query embeddings and resume chunk embeddings?
- **Q070:** What is Top-K retrieval, and how do you choose how many chunks to pass into the prompt context?
- **Q071:** How does Hirely's Substring Citation Verification eliminate hallucinated recruiter evidence?
- **Q072:** What happens if the recruiter asks a question about a skill completely absent from the resume?

Category 10: Prompt Engineering & Output Validation (Q73 - Q80)

- **Q073:** What are system instructions, user context, and task instructions, and how are they separated?
- **Q074:** How does Hirely implement context bounding to prevent prompt bloat and irrelevant completions?
- **Q075:** What is Few-Shot prompting vs Zero-Shot prompting, and which approach does Hirely employ?
- **Q076:** How do you instruct a language model to return "No evidence found" instead of guessing?
- **Q077:** How does Pydantic v2 validate LLM JSON completions, and what happens if a field is missing?
- **Q078:** What is Prompt Injection, and how does Hirely prevent candidates from hacking the matching prompts?
- **Q079:** Why is temperature set to a low value (e.g., 0.1 - 0.2) in Hirely's RAG and ATS pipelines?
- **Q080:** How do you test prompt templates for regression when modifying system instructions?

Category 11: AI Evaluation, Latency, Tokens & Cost (Q81 - Q88)

- **Q081:** How does Hirely's offline evaluation framework (`app.ai.eval.runner`) work?
- **Q082:** What metrics does the evaluation framework measure (retrieval precision, citation fidelity)?
- **Q083:** Why is offline evaluation essential in CI/CD without calling live billable APIs?
- **Q084:** How does Hirely extract token usage metadata from Gemini completions?
- **Q085:** How is estimated operational cost in USD calculated for each AI invocation?
- **Q086:** What are the average latencies for embedding generation, RAG retrieval, and full ATS analysis?
- **Q087:** How does Hirely track rolling *p50, p90*, and *p95* operational latencies without external APM tools?
- **Q088:** What strategies can you employ to reduce token consumption and operational costs at scale?

Category 12: Security & Reliability Engineering (Q89 - Q96)

- **Q089:** What is a ZIP-bomb attack, and how does Hirely's PDF file validator mitigate it?
- **Q090:** What are magic bytes, and why is checking `file.content_type` alone dangerous?
- **Q091:** How does Hirely's in-memory sliding-window rate limiter work under the hood?
- **Q092:** What are the limitations of process-local in-memory rate limiters in a multi-worker deployment?
- **Q093:** How does Hirely enforce dynamic CORS rules, and why is wildcard credentials forbidden?
- **Q094:** How does the backend trace requests across layers using correlation IDs (`X-Request-ID`)?
- **Q095:** How does the structured JSON logger protect candidate privacy and scrub PII?
- **Q096:** What is the Dual Storage architecture, and why was Cloudflare R2 integrated alongside local disk?

Category 13: Docker & Container Orchestration (Q97 - Q104)

- **Q097:** Explain the four orchestrated services defined in Hirely's `docker-compose.yml`.
- **Q098:** Why is PostgreSQL exposed on host port `5433` instead of the default `5432`?
- **Q099:** How does the `migration` container guarantee zero schema race conditions on stack startup?
- **Q100:** Walk me through the multi-stage build in `backend/Dockerfile` and why a non-root user is used.
- **Q101:** Walk me through the multi-stage build in `frontend/Dockerfile` and the role of Nginx.
- **Q102:** What are Docker named volumes, and how do `postgres_data` and `resume_uploads` persist data?
- **Q103:** How do container health checks work, and what commands are used for backend, frontend, and db?
- **Q104:** Why does a healthy Docker Compose stack not automatically mean an application is production-ready?

Category 14: Testing, Debugging & Trade-Offs (Q105 - Q112)

- **Q105:** How is Hirely's Pytest suite structured, and how are unit tests separated from integration tests?

- **Q106:** How does `conftest.py` ensure in-memory state (rate limiters, caches) is isolated between tests?
- **Q107:** How do you test external AI calls without burning Gemini API credits or failing on network drops?
- **Q108:** What is the difference between regression testing and evaluation benchmarking in AI applications?
- **Q109:** What CI checks are automated in GitHub Actions on every push to `main`?
- **Q110:** Describe a difficult bug you encountered during development and how you diagnosed and fixed it.
- **Q111:** What trade-offs were involved in using synchronous Docling parsing versus Celery worker queues?
- **Q112:** What trade-offs were involved in calculating vector cosine similarity in Python vs using `pgvector`?

Category 15: Limitations, Technical Debt & Ownership (Q113 - Q120)

- **Q113:** What are the primary architectural bottlenecks of Hirely in its current implementation?
- **Q114:** What technical debt currently exists in the backend or AI layer?
- **Q115:** How would you re-architect Hirely to support 100,000 active candidates and 1,000 concurrent recruiters?
- **Q116:** How would you introduce a dedicated vector database (e.g. pgvector or Pinecone) into the current schema?
- **Q117:** How would you add conversational multi-turn memory to the RAG Candidate Evidence Assistant?
- **Q118:** What was your exact personal role and contribution to the Hirely project?
- **Q119:** What parts of the codebase did you personally write, and what parts were collaborative or pre-existing?
- **Q120:** If you had another month to work on Hirely, what three features or architectural changes would you prioritize?